

Socket 编程实验文档

殷智宽 2024011029

2026 年 6 月 10 日

目录

1	引言	2
2	问题与应用场景	2
2.1	痛点分析	2
2.2	未来应用场景	2
3	协议概览	3
3.1	协议头整体设计	3
3.2	协议头各个组成部分设计	3
3.3	其他相关程序设计	3
4	核心交互与状态机	4
4.1	交互特点	4
4.2	状态机设计	4
5	运行方法	5
6	AI 使用情况	6
7	测试用例（相同内容可见视频文件）	6

1 引言

最初，我将这个大作业命名为：**Secure Cloud Clipboard** 即一种安全云剪贴板。但采纳了 AI 的建议后，我将其命名为：**Lightweight Cloud Encrypted Clipboard Protocol** 即 LCECP，设计目标是为开发者与学生提供一个在多设备间同步敏感凭证时的轻量级、安全且易调试的解决方案，避免使用商业通讯工具带来的隐私泄露与格式破坏问题。

简单来说，就是一个基于 TCP Socket 的轻量级协议，允许用户在多设备间安全地传递敏感凭证（如 GitHub Token、服务器 SSH 密钥、临时 API Key、代码片段等）。

由于直接通过微信等商业软件传递敏感数据存在隐私泄露风险，且格式易被打乱，因此我设计了一个专用的协议来解决这个问题。

核心是客户端在上传前利用本地密钥对凭证进行加密，服务器只存储和中转密文，对用户的真实数据一无所知，即使服务器被攻破，用户隐私也不会泄露。

2 问题与应用场景

2.1 痛点分析

在多设备（台式机、笔记本、服务器）间传递短文本凭证或代码片段时，常见问题包括：

- 数据被第三方平台存储或分析；
- 传输过程中明文泄露；
- 目标格式被平台处理后失真。

2.2 未来应用场景

本项目的设计目标是 최소화服务端对明文的可见性，仅中转密文并把解密逻辑保留在客户端，从而实现端到端加密。

因此，未来的应用场景包括但不限于：

- 开发者在不同开发环境（如本地 IDE、云服务器）间安全同步 API Key、SSH 密钥等敏感凭证；

- 学生在个人设备与实验室服务器间传递代码片段或配置文件；
- 以及其他任何需要在多设备间安全传输短文本数据的场景。

3 协议概览

3.1 协议头整体设计

为解决 TCP 字节流的粘包/拆包问题，协议定义了一个固定的 12 字节二进制协议头 Header，客户端/服务器通信流程为：先读取固定长度的 Header，再根据 Header 中的 `body_len` 字段读取后续正文 Body。协议头字段包括版本号、消息类型、状态码、正文长度和保留字段，便于后续扩展与兼容性维护。

具体协议头结构如下：

```
1 struct ProtocolHeader {  
2     uint8_t version;    // 协议版本  
3     uint8_t msg_type;   // 消息类型  
4     uint16_t status_code; // 状态码  
5     uint32_t body_len;   // 正文长度  
6     uint32_t reserved;  // 保留字段  
7 }; // 协议头结构体
```

3.2 协议头各个组成部分设计

对于协议版本，当前定义为 1，便于未来协议升级时进行兼容性判断；

对于消息类型，定义了 LOGIN、PUT、GET、EXIT 为客户端的四种基本操作，也定义了 RESP 为服务器端操作；

对于状态码，定义了 200 OK、400 Bad Request、413 Payload Too Large 等，以支持基本的状态返回和错误处理。

对于正文长度，设置了一个合理的上限即 64 KB，以防止恶意客户端发送过大的数据包导致服务器内存耗尽。

对于保留字段，目前未使用，默认为 0，预留给未来协议扩展使用。

3.3 其他相关程序设计

出于实验要求，本协议未设计复杂的 GUI，而是采用了纯控制台交互的方式，核心文件也仅为三个 python 程序：client.py、server.py 和 protocol.py，

工作量在千行以内。

对于加密算法的设计，思路是客户端采用 PBKDF2-HMAC-SHA256 将用户密码与随机生成的 128 位盐值进行密钥派生，生成 Fernet 对称加密密钥，对剪贴板内容进行端到端加密。服务器仅存储密文与盐值，不参与密钥生成和解密过程，因此无法获取用户明文数据；而客户端凭借正确的账户密码即可反解出明文，实现了基于 password 的端到端加密存储。

4 核心交互与状态机

4.1 交互特点

本协议采用典型的 client-server 模型，客户端通过 TCP Socket 连接服务器进行交互。而考虑到使用场景与协议复杂度，本协议数据传输限制为了半双工模式，即客户端在发送完请求后等待服务器响应，服务器在处理完请求后发送响应并等待下一次请求。

对于多用户场景，本协议通过 threading 库实现了对多用户并发连接的支持，但考虑到数据的安全与共享性，代码引入了全局线程锁，这一方面也导致了高并发场景下的弱支持度。

4.2 状态机设计

协议的核心交互流程可以抽象为一个状态机，由于协议设计的简洁性，状态机相对简单。只需考虑四种基本的客户端操作即可。

主要状态转移可见下图，其中虚线表示服务器的状态转移，实线表示客户端：

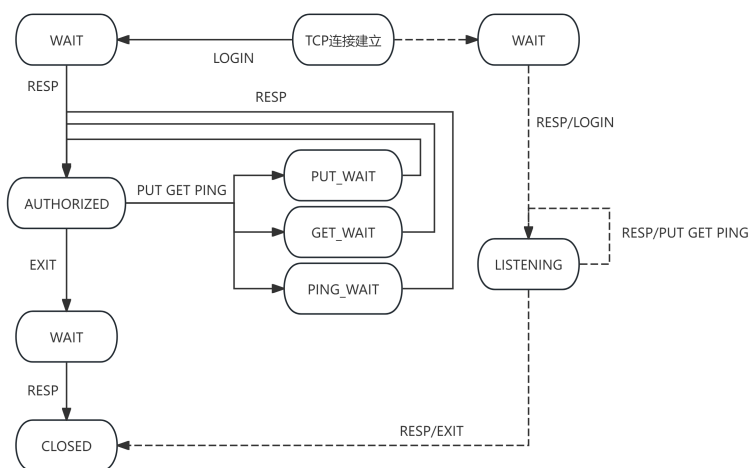


图 1: 协议状态机示意图

5 运行方法

由于本项目采用纯控制台交互方式，运行过程较为简单。在运行前，应确保客户端与服务器之间网络连通，并能够通过 TCP 协议建立连接。

1. 启动服务器：在服务器主机终端中运行 `python server.py`，服务器将在预设端口（默认 9000）启动监听，等待客户端连接。
2. 启动客户端：在客户端终端中运行 `python client.py`，根据程序提示输入用户名、密码以及相应操作编号即可完成登录、上传、获取等功能。
3. 客户端成功连接后，可通过菜单选择以下操作：
 - 1：上传加密后的剪贴板内容（PUT）；
 - 2：获取并解密剪贴板内容（GET）；
 - 3：发送心跳检测请求（PING）；
 - 4：退出客户端（EXIT）。

客户端会将输入的凭证加密后连同 salt 一同发送给服务器，服务器仅存储密文和 salt 并返回操作结果。用户可以在本地或其它设备上使用相同的客户端连接服务器获取之前上传的凭证。

6 AI 使用情况

在本项目中，AI 技术主要应用于以下几个方面：

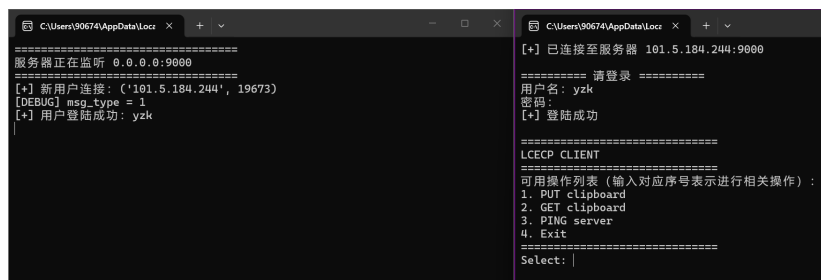
- 代码生成与优化：利用 AI 辅助生成加密相关算法，如 PBKDF2 密钥派生和 Fernet 加密的实现代码；
- 文本、文档撰写：通过 AI 协助设计实验文本如项目名称等和撰写实验文档，提高设计效率和文档质量；
- 错误检测与修复：通过 AI 模型检测代码中的潜在错误和安全漏洞，并提供优化或修复建议。

而项目的构思、协议设计、状态机设计等核心部分则由我独立完成。

7 测试用例（相同内容可见视频文件）

针对协议的核心功能，我设计了以下测试用例，主要以截图形式体现：

- 登录测试：验证客户端能否成功登录服务器，服务器能否正确处理登录请求并返回相应状态码。



```
=====
服务器正在监听 0.0.0.0:9000
=====
[+] 新用户连接: ('101.5.184.244', 19673)
[DEBUG] msg_type = 1
[+] 用户登陆成功: yzk

=====
[+] 已连接至服务器 101.5.184.244:9000
=====
请登录 =====
用户名: yzk
密码:
[+] 登陆成功

=====
LCECP CLIENT
=====
可用操作列表（输入对应序号表示进行相关操作）：
1. PUT clipboard
2. GET clipboard
3. PING server
4. Exit
=====
Select: |
```

图 2: 登录测试流程示意图

- PUT 操作测试：验证客户端能否成功上传敏感凭证，服务器能否正确存储密文并返回操作结果。

```
C:\Users\90674\AppData\Local\... x + v
=====
服务器正在监听 0.0.0.0:9000
=====
[+] 新用户连接: ('101.5.184.244', 19673)
[DEBUG] msg_type = 1
[+] 用户登陆成功: yzk
[DEBUG] msg_type = 2
[+] 密文上传成功! 账户yzk

C:\Users\90674\AppData\Local\... x + v
[+] 已连接至服务器 101.5.184.244:9000

===== 请登录 =====
用户名: yzk
密码:
[+] 登陆成功

=====
LCECP CLIENT
=====
可用操作列表 (输入对应序号表示进行相关操作):
1. PUT clipboard
2. GET clipboard
3. PING server
4. Exit
=====
Select: 1

===== 上传 =====
输入密文: 114514
[+] 上传成功!

=====
LCECP CLIENT
=====
可用操作列表 (输入对应序号表示进行相关操作):
1. PUT clipboard
2. GET clipboard
3. PING server
4. Exit
=====
Select:
```

图 3: PUT 操作测试流程示意图

- GET 操作测试: 验证客户端能否成功获取之前上传的凭证, 服务器能否正确返回密文。

```
C:\Users\90674\AppData\Local\... x + v
=====
服务器正在监听 0.0.0.0:9000
=====
[+] 新用户连接: ('101.5.184.244', 19673)
[DEBUG] msg_type = 1
[+] 用户登陆成功: yzk
[DEBUG] msg_type = 2
[+] 密文上传成功! 账户yzk
[DEBUG] msg_type = 3

C:\Users\90674\AppData\Local\... x + v
用户名: yzk
密码:
[+] 登陆成功

=====
LCECP CLIENT
=====
可用操作列表 (输入对应序号表示进行相关操作):
1. PUT clipboard
2. GET clipboard
3. PING server
4. Exit
=====
Select: 1

===== 上传 =====
输入密文: 114514
[+] 上传成功!

=====
LCECP CLIENT
=====
可用操作列表 (输入对应序号表示进行相关操作):
1. PUT clipboard
2. GET clipboard
3. PING server
4. Exit
=====
Select: 2

===== 获取 =====
解密内容: 114514
=====
```

图 4: GET 操作测试流程示意图

- 错误处理测试: 验证服务器能否正确处理超过正文长度限制的请求, 并返回 413 状态码。

```
===== 请登录 =====
用户名: 123
密码:
[+] 登陆成功

=====
LCECP CLIENT
=====
可用操作列表 (输入对应序号表示进行相关操作):
1. PUT clipboard
2. GET clipboard
3. PING server
4. Exit
=====
Select: 1

===== 上传 =====
输入密文: (此处以text = A * 60000为测例验证防大包攻击功能)
[!] Error: 数据量过大! 请减小数据量!
```

图 5: 错误处理测试流程示意图

- 多用户并发测试：验证服务器在多个客户端同时连接和操作时的稳定性和数据安全性。

```
=====
服务器正在监听 0.0.0.0:9000
[+] 新用户连接: ('101.5.184.244', 5492)
[+] 新用户连接: ('101.5.184.244', 5495)
[DEBUG] msg_type = 1
[+] 用户登陆成功: 123
[+] 用户登陆成功: 123
[+] 新用户连接: ('101.5.184.244', 5499)
[DEBUG] msg_type = 1
[+] 用户登陆成功: 114514
[DEBUG] msg_type = 2
[+] 密文上传成功! 密户114514
[DEBUG] msg_type = 3
[DEBUG] msg_type = 3
[DEBUG] msg_type = 5
[DEBUG] msg_type = 2
[+] 密文上传成功! 密户123
[DEBUG] msg_type = 3
|
```

服务端

```
2. GET clipboard
3. PING server
4. Exit
=====
Select: 2
===== 获取 =====
解密内容: 73737
=====
LCECP CLIENT
=====
可用操作列表 (输入对应序号表示进行相关操作):
1. PUT clipboard
2. GET clipboard
3. PING server
4. Exit
=====
Select: |
```

客户端1

```
2. GET clipboard
3. PING server
4. Exit
=====
Select: 2
===== 获取 =====
解密内容: 1919810
=====
LCECP CLIENT
=====
可用操作列表 (输入对应序号表示进行相关操作):
1. PUT clipboard
2. GET clipboard
3. PING server
4. Exit
=====
Select:
```

客户端2

```
2. GET clipboard
3. PING server
4. Exit
=====
Select: 2
===== 获取 =====
[!] 密文为空!
=====
LCECP CLIENT
=====
可用操作列表 (输入对应序号表示进行相关操作):
1. PUT clipboard
2. GET clipboard
3. PING server
4. Exit
=====
Select:
```

客户端3

图 6: 多用户交互流程示意图

- 跨主机测试：验证客户端和服务器在不同主机上运行时的网络连通性

和协议交互正确性。

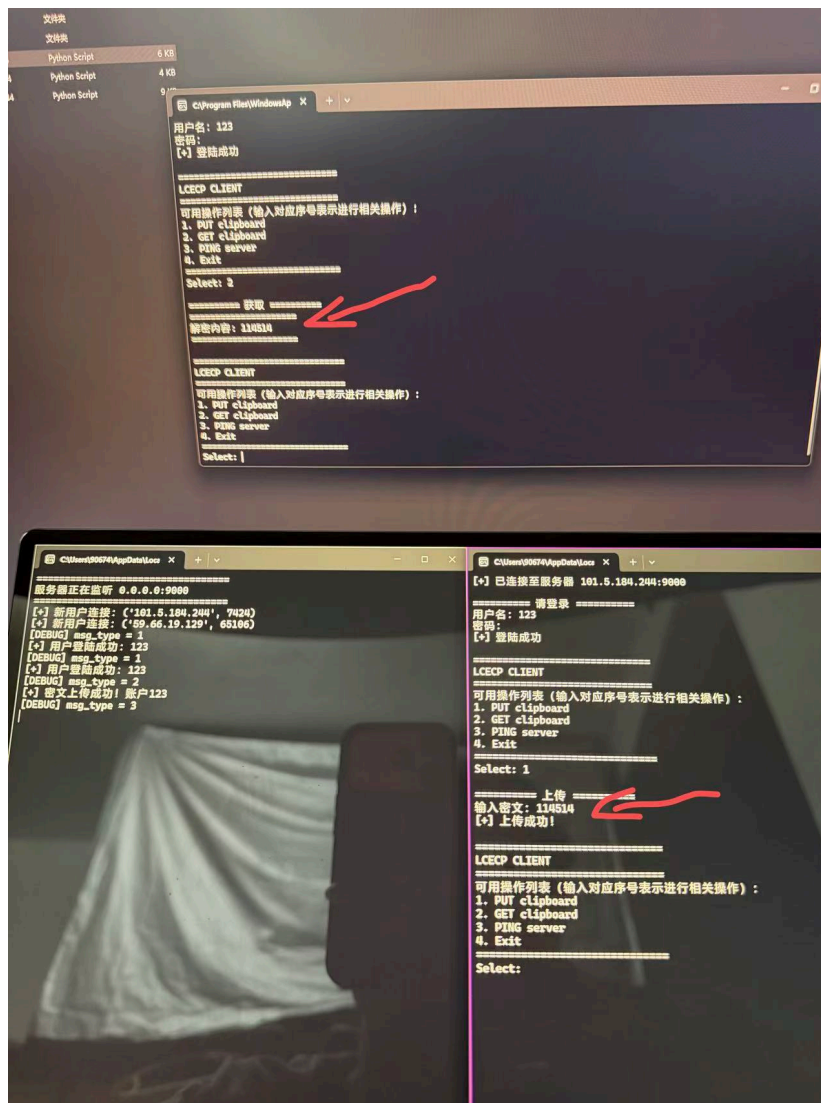


图 7: 跨主机测试流程示意图